NIFTY AI Trading Engine: Current Architecture and Business Logic

Last updated: 2026-05-10

1. Scope and current source of truth

This document is the current architecture and business-logic reference for the active engine in this repository.

It covers:
â¢ the current intraday defined-risk engine under ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk‘
â¢ the current weekly research-only engine under ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/weekly_defined_risk‘
â¢ the operational runtime, health, reconciliation, paper-mode learning layer, datasets, and artifacts

Important:
â¢ The old top-level ‘README.md‘ still describes an older weekly theta workflow and is not the full source of truth for the current intraday v83 stack.
â¢ The current approved live candidate is the frozen ‘v83‘ intraday bearish engine.
â¢ Weekly defined-risk logic is research-only.

2. Current approved system posture

Intraday v83

Validated benchmark artifact:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_post_tuesday_expiry_benchmark_2026-04-12_v83.json‘

Current approved benchmark metrics:
â¢ trades: 10
â¢ wins: 10
â¢ losses: 0
â¢ realized P&L: Rs 25,303.12
â¢ strategy used: ‘BEAR_CALL_CREDIT_SPREAD‘
â¢ playbook distribution:
â¢ ‘SIDEWAYS_TO_BEARISH_REJECTION‘: 7
â¢ ‘GAP_DOWN_BEARISH_CONTINUATION‘: 2
â¢ ‘GAP_UP_BEARISH_FAILURE‘: 1

Current runtime configuration artifact:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_runtime_config.json‘

Current approved live/paper rules:
â¢ mode currently configured as ‘PAPER_LIVE‘
â¢ live playbooks enabled:
â¢ ‘SIDEWAYS_TO_BEARISH_REJECTION‘

â¢ ‘GAP_DOWN_BEARISH_CONTINUATION‘
â¢ ‘GAP_UP_BEARISH_FAILURE‘
â¢ live strategy enabled:
â¢ ‘BEAR_CALL_CREDIT_SPREAD‘
â¢ live approved market states:
â¢ ‘TRANSITION‘
â¢ ‘TREND_DOWN‘
â¢ bullish live trading: disabled
â¢ condor live trading: disabled
â¢ defined-risk only: yes

Weekly defined-risk module

Current weekly research is not promoted.

Key research artifacts:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/weekly_defined_risk/weekly_nifty_defined_risk_backtest_summary_2026-04-28_v5_rebuilt_calendar.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/weekly_defined_risk/weekly_nifty_defined_risk_backtest_summary_2026-04-28_bearish_credit_only_rebuilt_calendar.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/weekly_defined_risk/weekly_nifty_vs_intraday_v83_2026-04-28_v5_rebuilt_calendar.json‘

Current weekly conclusion:
â¢ research only
â¢ not promoted to shadow/live
â¢ insufficient robustness and sample size versus intraday v83

3. Repository map

Current core engine paths

â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/weekly_defined_risk‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/utils/nifty_expiry_calendar.py‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/scripts‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/ui‘

Intraday module roles

â¢ ‘data_models.py‘: canonical dataclasses and enums
â¢ ‘features.py‘: price, structure, EMA, VWAP, candle, and option-chain features
â¢ ‘regime.py‘: market-state engine, regime classification, subtype tagging, metadata generation
â¢ ‘strategy.py‘: playbook routing, tradability, time windows, tiering
â¢ ‘strikes.py‘: multi-candidate spread construction and monetization scoring
â¢ ‘risk.py‘: lot sizing, max-loss calculations, margin controls
â¢ ‘execution.py‘: entry validation, open-position construction, exit logic
â¢ ‘decision.py‘: trade/no-trade decision agent helpers
â¢ ‘learning.py‘: SQLite learning store, playbook summaries, guarded calibration state

â¢ ‘backtest.py‘: full benchmark engine, funnel reports, subtype reports, opportunity reports
â¢ ‘dataset.py‘: structured dataset creation and validation
â¢ ‘research.py‘: research dataset creation and backtest optimization support
â¢ ‘pipeline.py‘: rolling -> structured -> research data refresh pipeline
â¢ ‘collector.py‘: live decision-time option-chain snapshot collector
â¢ ‘data_pipeline_health.py‘: freshness and health checks for datasets and collectors
â¢ ‘live_runtime.py‘: Dhan-backed live market snapshot provider and execution adapter support
â¢ ‘ops_runtime.py‘: runtime config, health model, entry gates, reconciliation, reporting
â¢ ‘monitor.py‘: live loop, paper mode, shadow mode, operational orchestration
â¢ ‘cli.py‘: CLI entry point for all core intraday operations

Weekly module roles

â¢ ‘weekly_defined_risk/research.py‘: weekly dataset builder, regime classifier, structure generation, exit-grid backtest, weekly vs intraday comparison
â¢ ‘scripts/run_weekly_defined_risk_research.py‘: weekly research CLI wrapper
â¢ ‘utils/nifty_expiry_calendar.py‘: official weekly-expiry inference and holiday alignment

4. High-level architecture

```
flowchart TD
A["Rolling / structured market data"] --> B["Research dataset builders"]
A --> C["Live market snapshot provider"]
B --> D["Intraday backtest + benchmark engine"]
C --> E["Intraday v83 live monitor"]
E --> F["Regime + state engine"]
F --> G["Playbook routing + tradability layer"]
G --> H["Defined-risk spread construction"]
H --> I["Risk and health gates"]
I --> J["Mode-specific execution path"]
J --> K["Shadow logs / paper trades / broker execution"]
D --> L["Benchmark artifacts + subtype research reports"]
B --> M["Weekly defined-risk research engine"]
M --> N["Weekly dataset + weekly backtest artifacts"]
```

5. Canonical data contracts

The core dataclasses are defined in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/data_models.py‘

Important contracts:
â¢ ‘MarketSnapshot‘
â¢ ‘OhlcvBar‘
â¢ ‘OhlcvSeries‘
â¢ ‘OptionsContractQuote‘
â¢ ‘OptionsChainSnapshot‘
â¢ ‘TradeStructure‘
â¢ ‘DecisionOutput‘
â¢ ‘OpenPosition‘

â¢ ‘ExitDecision‘
â¢ ‘AdaptiveParameters‘
â¢ ‘RegimeState‘

Core enums:
â¢ ‘OptionType‘
â¢ ‘StrategyType‘
â¢ ‘RegimeLabel‘

Business meaning:
â¢ ‘MarketSnapshot‘ is the atomic input to the live or backtest decision engine.
â¢ ‘RegimeState‘ is the rich state-layer output from ‘regime.py‘.
â¢ ‘DecisionOutput‘ is the final instruction object passed into runtime gating and paper/live orchestration.
â¢ ‘TradeStructure‘ guarantees defined-risk structure metadata before entry.

6. Intraday data architecture

6.1 Structured training dataset

Primary builder and validator:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/dataset.py‘

Expected files in a structured dataset root:
â¢ ‘nifty_5m.csv‘
â¢ ‘option_chain_decision_times.csv‘
â¢ ‘execution_fills.csv‘
â¢ ‘session_labels.csv‘

Business requirements enforced in code:
â¢ required decision-time snapshots: ‘09:30‘, ‘10:00‘, ‘13:00‘
â¢ minimum training coverage: 60 trading days
â¢ target training coverage: 120 trading days
â¢ expected session labels:
â¢ ‘trend_down‘
â¢ ‘trend_up‘
â¢ ‘range‘
â¢ ‘event_day‘

Structured dataset coverage checks include:
â¢ 5-minute OHLCV coverage
â¢ chain snapshot coverage
â¢ delta coverage
â¢ IV coverage
â¢ bid/ask coverage
â¢ realized fill coverage
â¢ label coverage

6.2 Research dataset

Research dataset builders:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/research.py‘

Research dataset outputs typically include:

â¢ ‘nifty_5m.csv‘
â¢ ‘nifty_15m.csv‘
â¢ ‘options_chain.csv‘

Business logic:
â¢ derive 15-minute bars from 5-minute bars
â¢ assign weekly expiry using ‘infer_nifty_weekly_expiry‘
â¢ derive delta with Black-Scholes when missing
â¢ derive synthetic bid/ask when historical bid/ask is missing
â¢ prepare dense benchmark-ready option-chain rows for backtest replay

6.3 Automated refresh pipeline

Pipeline orchestrator:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/pipeline.py‘

Pipeline responsibilities:
â¢ fetch missing rolling history
â¢ refresh structured dataset from rolling history
â¢ enrich missing deltas
â¢ append/refresh research dataset
â¢ write training pipeline status

State/status artifact:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_training_pipeline_st
atus.json‘

7. Intraday feature and state engine

7.1 Price and structure features

Feature generation lives primarily in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/features.py‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/regime.py‘

Important feature families used by the current engine:
â¢ 5-minute execution structure
â¢ 15-minute trend structure
â¢ EMA20 / EMA50 / EMA100 alignment, slope, spacing, compression
â¢ VWAP relation and acceptance
â¢ opening-range relation
â¢ candle overlap and trend efficiency
â¢ BOS / CHoCH style structure shifts
â¢ failed breakout / failed bounce / failed reclaim / accepted breakdown
â¢ support and resistance distance
â¢ open space to next level
â¢ option-chain pressure
â¢ call wall / put wall
â¢ OI pressure and wall migration
â¢ candle-pattern quality and rejection quality

7.2 Market state engine

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/regime.py‘

Current market states:
â¢ ‘TREND_UP‘
â¢ ‘TREND_DOWN‘
â¢ ‘TRUE_RANGE‘
â¢ ‘DIRECTIONAL_BALANCE‘
â¢ ‘TRANSITION‘

Business meaning:
â¢ ‘TREND_DOWN‘: aligned bearish trend with continuation characteristics
â¢ ‘TREND_UP‘: aligned bullish trend with continuation characteristics
â¢ ‘TRUE_RANGE‘: unstable range / chop / low directional acceptance
â¢ ‘DIRECTIONAL_BALANCE‘: compressed or balanced session with directional bias and possible bearish or bullish resolution
â¢ ‘TRANSITION‘: highest-priority change-of-structure / failure / rejection regime

Important design point:
â¢ ‘TRANSITION‘ is intentionally treated as a high-information state.
â¢ ‘TRUE_RANGE‘ is usually blocked for directional trading.
â¢ ‘DIRECTIONAL_BALANCE‘ was introduced to avoid over-penalizing all balanced sessions as no-trade.

7.3 Tradability layer

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strategy.py‘

Tradability classes:
â¢ ‘TRADABLE‘
â¢ ‘LOW_EDGE‘
â¢ ‘NOT_TRADABLE‘

Business meaning:
â¢ ‘TRADABLE‘: eligible for spread monetization under the validated stack
â¢ ‘LOW_EDGE‘: structurally visible, but not strong enough for broad deployment
â¢ ‘NOT_TRADABLE‘: detected but blocked before spread construction because edge or monetization is structurally unproven

Current key mappings:
â¢ stable bearish spread playbooks are ‘TRADABLE‘
â¢ late bullish reclaim remains ‘LOW_EDGE‘
â¢ bullish expansion playbooks are ‘NOT_TRADABLE‘ under the current spread framework

8. Playbooks, tiers, and business logic

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strategy.py‘

8.1 Playbook tiers

Tier A:

â¢ ‘SIDEWAYS_TO_BEARISH_REJECTION‘
â¢ ‘GAP_DOWN_BEARISH_CONTINUATION‘
â¢ ‘GAP_UP_BEARISH_FAILURE‘
â¢ ‘RANGE_BALANCED_CONDOR‘

Tier B:
â¢ research-only bullish and secondary bearish families
â¢ includes bullish reclaim/continuation variants and several bearish research families

Tier C:
â¢ detect-only / unsupported / residual routing states

8.2 Current approved live subset

Even though ‘RANGE_BALANCED_CONDOR‘ is Tier A historically, the current runtime configuration only enables:
â¢ ‘SIDEWAYS_TO_BEARISH_REJECTION‘
â¢ ‘GAP_DOWN_BEARISH_CONTINUATION‘
â¢ ‘GAP_UP_BEARISH_FAILURE‘

The runtime currently enables only:
â¢ ‘BEAR_CALL_CREDIT_SPREAD‘

That is an intentional operator/runtime restriction on top of the broader research system.

8.3 Key bearish families implemented

The engine has explicit or research-level handling for:
â¢ failed reclaim
â¢ failed bounce
â¢ failed breakout
â¢ accepted breakdown
â¢ gap-down continuation / failed bounce
â¢ sideways bearish rejection
â¢ lower-high failure behavior
â¢ OR-low retest failure
â¢ supply-zone failed reclaim
â¢ transition rejection family

8.4 Bullish logic status

Bullish logic exists in the codebase for detection and research, but not for live deployment.

Important business conclusion already embedded into the architecture:
â¢ bullish contexts are detectable
â¢ bullish spread monetization has not been validated for live use
â¢ bullish live routing remains blocked

9. Strategy routing and spread monetization

9.1 Strategy routing

Routing logic spans:

â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strategy.py‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/decision.py‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/monitor.py‘

Current live route:
â¢ bearish context -> ‘BEAR_CALL_CREDIT_SPREAD‘

Research routes supported in the codebase:
â¢ ‘BULL_PUT_CREDIT_SPREAD‘
â¢ ‘CALL_DEBIT_SPREAD‘
â¢ ‘IRON_CONDOR‘
â¢ shadow-only bullish and bearish families

9.2 Multi-candidate spread construction

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strikes.py‘

Business logic:
â¢ the engine evaluates multiple widths instead of a single rigid spread
â¢ directional width candidates include ‘50‘, ‘75‘, ‘100‘, ‘150‘
â¢ condor width candidates include multiple symmetric and asymmetric combinations
â¢ candidates are scored on monetization quality
â¢ best valid candidate is selected
â¢ failed candidates keep explicit rejection reasons

9.3 Monetization filters

Core directional rejection reasons include:
â¢ ‘LIQUIDITY_BAD‘
â¢ ‘DELTA_TOO_HIGH‘
â¢ ‘INVALIDATION_TOO_CLOSE‘
â¢ ‘CREDIT_TOO_LOW‘
â¢ ‘HEDGE_TOO_EXPENSIVE‘
â¢ ‘WIDTH_TOO_LARGE‘
â¢ ‘NET_EDGE_TOO_LOW‘

Directional business rules include:
â¢ short strike must not be too close to invalidation
â¢ credit/width must clear strategy-specific threshold
â¢ hedge cost must remain acceptable
â¢ quote spread quality must be acceptable
â¢ net edge after assumed cost must remain positive enough

9.4 Adaptive parameters

Adaptive parameters are defined in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/data_models.p
y‘

They include:
â¢ score thresholds

â¢ credit/width thresholds
â¢ hedge-cost caps
â¢ liquidity caps
â¢ width preferences
â¢ shadow-session count
â¢ bearish/bullish trade thresholds and margins

Important point:
â¢ the parameter system exists, but the runtime still keeps the frozen v83 strategy stack and guarded promotion model.

10. Risk model and exit logic

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/risk.py‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/execution.py‘

10.1 Core constraints

Non-negotiable controls in the current engine:
â¢ defined-risk only
â¢ no naked option selling
â¢ no uncovered ratio structures in promoted logic
â¢ max loss known before entry
â¢ risk-based lot sizing
â¢ margin-based lot sizing
â¢ kill switch on realized daily loss and extreme margin utilization

10.2 Entry-time controls

Entry constraints include:
â¢ no entries before 09:15 IST
â¢ directional entries prefer 09:30 onward
â¢ directional entries blocked after 14:00 IST in strict v83
â¢ time exit enforced by 15:15 IST
â¢ current runtime also has ‘no_new_entries_after_hhmm‘ in risk governance

10.3 Exit model

Current exit engine supports:
â¢ time exit
â¢ premium stop
â¢ delta stop
â¢ regime invalidation
â¢ session profit trail
â¢ structure profit trail
â¢ bullish-specific relaxed delta stop for selected research playbooks

Business meaning:
â¢ the exit engine is shared by backtest, paper, and live orchestration
â¢ the paper override path did not change exits; it only changes which paper candidates can be simulated

11. Learning, calibration, and benchmark research

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/learning.py‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/backtest.py‘

11.1 Learning store

Learning store DB:
â¢ default path: ‘/tmp/intraday_defined_risk_learning.sqlite3‘
â¢ paper runtime currently also uses artifacts under
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_paper_live_learn
ing.sqlite3‘

Tables:
â¢ ‘decisions‘
â¢ ‘outcomes‘
â¢ ‘parameter_state‘
â¢ ‘parameter_history‘

Business purpose:
â¢ log decisions and outcomes
â¢ compute playbook summaries
â¢ run guarded parameter shadow calibration
â¢ avoid promoting candidates without objective improvement and drawdown control

11.2 Benchmark engine

Backtest engine:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/backtest.py‘

Outputs include:
â¢ trade funnel reports
â¢ subtype reports
â¢ rejection reason counts
â¢ regime tradability reports
â¢ market-state reports
â¢ monthly missed opportunity summaries
â¢ opportunity benchmark vs strict benchmark comparisons

Business philosophy embedded in the benchmark code:
â¢ do not optimize for 100 percent win rate alone
â¢ separate setup quality from monetization quality
â¢ log what was seen, what was rejected, and why
â¢ compare strict live-eligible stack against broader opportunity layers

12. Runtime modes and operational controls

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/ops_runtime.p
y‘

â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/monitor.py‘

12.1 Runtime modes

Supported runtime modes:
â¢ ‘RESEARCH‘
â¢ ‘SHADOW_LIVE‘
â¢ ‘PAPER_LIVE‘
â¢ ‘MICRO_LIVE‘
â¢ ‘LIVE_DISABLED‘

Business meaning:
â¢ ‘RESEARCH‘: offline/backtest/research evaluation only
â¢ ‘SHADOW_LIVE‘: live scoring, no positions
â¢ ‘PAPER_LIVE‘: simulated positions, same risk and exit engine, no broker orders
â¢ ‘MICRO_LIVE‘: real broker execution, blocked unless armed
â¢ ‘LIVE_DISABLED‘: hard block

12.2 Runtime risk governance

Current governance keys:
â¢ ‘max_lots_per_trade‘
â¢ ‘max_open_structures‘
â¢ ‘max_trades_per_day‘
â¢ ‘max_daily_realized_loss_rupees‘
â¢ ‘max_daily_total_loss_rupees‘
â¢ ‘no_new_entries_after_hhmm‘
â¢ ‘stop_after_first_full_loss‘
â¢ ‘require_manual_live_arm‘
â¢ ‘live_only_bearish‘

Current configured values in runtime config:
â¢ max lots per trade: 1
â¢ max open structures: 1
â¢ max trades per day: 1
â¢ no new entries after: 14:00
â¢ manual live arm required: yes
â¢ bearish only: yes

12.3 Entry gate model

The runtime entry gate verifies:
â¢ mode allows entry
â¢ candidate is ‘TRADE‘
â¢ playbook is live-enabled
â¢ strategy is live-enabled
â¢ tradability is not ‘NOT_TRADABLE‘
â¢ market state is approved
â¢ bearish score clears required margin over no-trade score
â¢ time window is open
â¢ daily locks are inactive
â¢ no active structure already exists
â¢ health model is not blocked
â¢ reconciliation/recovery state is clean

13. Unified health model

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/ops_runtime.p
y‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/data_pipeline
_health.py‘

Health statuses:
â¢ ‘HEALTHY‘
â¢ ‘DEGRADED‘
â¢ ‘BLOCKED‘

Components tracked:
â¢ ‘broker_auth_health‘
â¢ ‘market_feed_health‘
â¢ ‘option_chain_health‘
â¢ ‘broker_position_sync_health‘
â¢ ‘state_store_health‘
â¢ ‘strategy_engine_health‘
â¢ ‘data_pipeline_health‘

Primary operator artifact:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_operator_status_
report.json‘

This report includes:
â¢ runtime config
â¢ runtime state
â¢ health summary
â¢ reconciliation summary
â¢ recovery state
â¢ paper and shadow summaries
â¢ validation anomalies
â¢ promotion gates

14. Reconciliation, recovery, and emergency controls

Implemented in:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/ops_runtime.p
y‘

Reconciliation statuses:
â¢ ‘MATCHED‘
â¢ ‘NO_POSITIONS‘
â¢ ‘POSITION_MISMATCH‘
â¢ ‘ORPHAN_POSITION‘
â¢ ‘ORPHAN_STATE‘
â¢ ‘UNKNOWN‘

Business behavior:

â¢ mismatches or orphan states block new entries
â¢ paper mode intentionally treats absence of broker positions as normal
â¢ micro-live is stricter and expects broker/internal consistency

Emergency functionality:
â¢ flatten-all routine exists
â¢ recovery state is activated on emergency flatten
â¢ new entries remain blocked until recovery is cleared

Key artifacts:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_reconciliation_status.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_reconciliation_events.jsonl‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_operator_events.jsonl‘

15. Paper-live learning layer and experimental override

15.1 Baseline paper problem

Observed operationally:
â¢ paper mode was healthy
â¢ strict v83 often emitted ‘NO_TRADE‘
â¢ no paper positions were created because routing was too strict for learning purposes

15.2 Paper-only override

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/monitor.py‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/ops_runtime.py‘

Current paper-only learning path:
â¢ ‘PAPER_CONTEXT_OVERRIDE‘

Business purpose:
â¢ allow paper-mode simulation of high-confidence bearish near-miss contexts
â¢ do not change strict v83 live logic
â¢ do not change ‘MICRO_LIVE‘
â¢ keep spread construction, exits, and risk model unchanged

Eligibility rules for paper override include:
â¢ mode must be ‘PAPER_LIVE‘
â¢ state in ‘TRANSITION‘, ‘TREND_DOWN‘, or ‘DIRECTIONAL_BALANCE‘
â¢ ‘tradability_class == TRADABLE‘
â¢ failure type in:
â¢ ‘FAILED_RECLAIM‘
â¢ ‘FAILED_BOUNCE‘
â¢ ‘FAILED_BREAKOUT‘
â¢ ‘ACCEPTED_BREAKDOWN‘

â¢ bearish score >= paper override threshold
â¢ bearish score > no-trade score + paper override margin
â¢ no ‘TRUE_RANGE‘
â¢ no bullish context
â¢ no active paper structure
â¢ health gate must pass
â¢ within paper experiment time window

Current paper override artifacts:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/paper_context_override_report
.json‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/near_trade_candidates.csv‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/paper_time_window_audit.json‘

Important note:
â¢ historical validation rows generated before the override was added are marked as legacy and require replay for true override performance statistics

15.3 Decision attribution

Paper decisions are separated into:
â¢ ‘V83_APPROVED‘
â¢ ‘PAPER_CONTEXT_OVERRIDE‘

This is important because the paper learning layer must not pollute strict v83 performance accounting.

16. Operational files and reports

Core runtime files

â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_runtime_config.j
son‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_runtime_state.js
on‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_run_live_config.
json‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_runner.log‘

Paper and shadow reports

â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_shadow_live_repo
rt.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_paper_live_repor
t.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_paper_live_valid

ation_report.json‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/intraday_v83_paper_live_valid
ation_decisions.jsonl‘

New paper override reports

â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/paper_context_override_report
.json‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/near_trade_candidates.csv‘
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/paper_time_window_audit.json‘

Weekly research outputs

â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/weekly_defined_risk/*.json‘
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/state/weekly_defined_risk/*.csv‘

17. Weekly defined-risk research engine

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/weekly_defined_risk/research.py‘

17.1 Objective

Research whether weekly holding strategies can improve trade frequency or risk-adjusted return versus intraday v83.

This module is explicitly separate from the intraday runtime path.

17.2 Weekly dataset model

Each week context includes:
â¢ deployment date
â¢ deployment weekday
â¢ preferred next weekly expiry
â¢ observed historical expiry
â¢ spot
â¢ Monday gap percent
â¢ weekly ATR
â¢ daily ATR
â¢ proxy India VIX / ATM IV
â¢ previous week high / low / close
â¢ daily trend state
â¢ 60-minute trend state
â¢ option-chain snapshot summary
â¢ call wall / put wall / PCR / OI pressure
â¢ support / resistance
â¢ event flags
â¢ missing-data flags
â¢ trade-ready and no-trade reason

17.3 Weekly regimes

Current weekly regimes:
â¢ ‘BULLISH_CONTINUATION‘
â¢ ‘BEARISH_CONTINUATION‘
â¢ ‘SIDEWAYS_RANGE‘
â¢ ‘HIGH_VOL_EVENT‘
â¢ ‘GAP_UP_FAILURE‘
â¢ ‘GAP_DOWN_RECOVERY‘
â¢ ‘LOW_EDGE_NO_TRADE‘

17.4 Weekly structures researched

Defined-risk only:
â¢ ‘BULL_PUT_CREDIT_SPREAD‘
â¢ ‘BEAR_CALL_CREDIT_SPREAD‘
â¢ ‘IRON_CONDOR‘
â¢ ‘BROKEN_WING_CALL_FLY‘
â¢ ‘BROKEN_WING_PUT_FLY‘
â¢ ‘NO_TRADE‘

Business constraints:
â¢ no naked shorts
â¢ no uncovered ratio structures
â¢ max loss must be known before entry

17.5 Weekly expiry calendar handling

Implemented in:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/utils/nifty_expiry_calendar.py‘

Current logic:
â¢ legacy weekly expiry weekday: Thursday
â¢ current weekly expiry weekday after 2025-09-01 regime change: Tuesday
â¢ expiry is aligned to prior trading day when the nominal expiry day is absent due to holiday or dataset gap

17.6 Weekly research conclusion

The weekly engine exists and runs, but it remains research-only because:
â¢ sample size is small
â¢ robustness is weak
â¢ intraday v83 remains superior on validated evidence
â¢ weekly promotion has not cleared the current promotion bar

18. CLI entry points

Primary intraday CLI:
â¢ ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/cli.py‘

Key supported commands include:
â¢ ‘run_live‘
â¢ ‘run_backtest‘
â¢ ‘calibrate‘
â¢ ‘validate_dataset‘
â¢ ‘scaffold_dataset‘

â¢ ‘build_structured_dataset‘
â¢ ‘collect_chain_snapshot‘
â¢ ‘run_chain_schedule‘
â¢ ‘run_daily_chain_schedule‘
â¢ ‘enrich_chain_deltas‘
â¢ ‘build_research_dataset‘
â¢ ‘build_research_dataset_from_rolling‘
â¢ ‘append_research_dataset_from_rolling‘
â¢ ‘refresh_structured_training_dataset‘
â¢ ‘refresh_training_data‘
â¢ ‘run_training_data_pipeline‘
â¢ ‘optimize_backtest‘
â¢ ‘ops_status‘
â¢ ‘set_runtime_mode‘
â¢ ‘flatten_all‘
â¢ ‘write_operational_reports‘

Weekly research CLI:
â¢
‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/scripts/run_weekly_defined_risk_res
earch.py‘

19. Current business logic summary

What the engine is trying to do

The active system is not a generic pattern trader.
It is a context-aware defined-risk options engine that works in this order:
â¢ build market snapshot
â¢ classify state and structure
â¢ classify tradability
â¢ choose playbook
â¢ choose strategy
â¢ construct best valid spread candidate
â¢ pass risk and runtime gates
â¢ enter only if the opportunity is both structurally valid and monetizable

What is currently validated

Validated for strict v83:
â¢ bearish intraday defined-risk credit spreads
â¢ strongest on transition / trend-down bearish contexts
â¢ especially through the three approved live bearish playbooks

What is intentionally not promoted

Not promoted live:
â¢ bullish live trading
â¢ condor live routing
â¢ weekly engine
â¢ broad paper override results
â¢ broad failed-breakout family promotion
â¢ blanket loosening of thresholds

What the architecture is optimized for

The codebase is optimized for:
â¢ honest opportunity logging
â¢ separation of structure from monetization
â¢ state-aware routing
â¢ exact rejection attribution
â¢ guarded runtime modes
â¢ strong operator visibility
â¢ defined-risk enforcement

20. Known constraints and current gaps

Current operational caveats

The operator status report can legitimately show ‘BLOCKED‘ on non-trading days or stale structured data days.
That does not necessarily imply a strategy bug; it can be a freshness gate doing its job.

Current paper-learning caveat

The new ‘PAPER_CONTEXT_OVERRIDE‘ path exists, but historical rows generated before its implementation cannot retroactively prove its performance without replay.

Current weekly caveat

The weekly research engine has been corrected for expiry calendar handling, but it still does not have enough robust evidence to outrank intraday v83.

21. Recommended reading order in the code

If you want to understand the engine quickly, read in this order:

1. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/data_models.py‘
2. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/regime.py‘
3. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strategy.py‘
4. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/strikes.py‘
5. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/risk.py‘
6. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/execution.py‘
7. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/monitor.py‘
8. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/ops_runtime.py‘
9. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/intraday_defined_risk/backtest.py‘
10. ‘/Users/Rohit/AI-Trading-Engine/data_engine/market_ai/weekly_defined_risk/research.py‘

22. Summary

The implemented system today is best understood as two separate layers:

1. A frozen intraday v83 bearish defined-risk production candidate with strict runtime controls.
2. A research framework around it for dataset building, benchmarking, subtype mining, paper-only override learning, and weekly defined-risk exploration.

That separation is deliberate.
The production path stays narrow and controlled.
The research path is broad, diagnostic, and attribution-heavy.